OPERATING PROCEDURE · TEAM 视角 · 岗位 · 四个载体

超级智能体上手作业指导书

Team 视角（PM 岗位）怎么带项目、岗位怎么接活——
一份照着做的日常流程：宪章 → 任务 → 消息 → 资产，配合面板「编队」页。

Team 视角 · 管流程 | 岗位 · 干职责 | Kanban · 任务 | Gitea · 代码 | ntfy · 消息 | MinIO · 资产

读者
Team 视角（PM 岗位）/ Worker 岗位的负责人

配套文档
SOP.md · RD.md · STORY.md

面板位置
编队 · 设置→项目与岗位

SOP

ROLES

两个角色：岗位与岗位职责

ROLE 01

岗位（post）

一个 SuperAgent 实例，长期存在，**持有外部身份**：Kanban 账号、Gitea 账号、ntfy 身份。

扮演 PM 或 Worker，负责一个项目里的一个角色。

长期 · 有身份 · 一个实例

ROLE 02

岗位职责（duty）

岗位内部**一次具体任务的一次执行**，没有外部身份。

随工单开始，随交付结束；WIP 统计的对象就是它。

一次性 · 无身份 · 一个任务

角色	一句话职责	主要动作	主战场
PM 岗位	让项目按流程跑起来	颁布宪章 · 派工 · 审进度 · 处理异常	面板「编队」页
岗位（Worker）	把被派到的职责做完	接单 · 执行 · 交付 · 回写看板	面板「执行」页

PANORAMA

一分钟全景：一条卡怎么变成交付

Kanban 卡片
任务来源
→
指派给岗位
assignee = 岗位账号
→
岗位接单
读回 + 租约
→
工单（职责）
进入执行队列
→
执行会话
写代码 / 文档 / memo
→
回写看板
状态 + 日志 + 引用

Kanban 是唯一任务源

只有看板上的卡才是任务；评论 @ 岗位不算派工。

Gitea 是代码资产

代码变更的权威载体是 PR；一仓一目录一 PR。

ntfy 是消息通道

实时互通与告警；公开 topic 只作上下文，不构成授权。

MinIO 是扩展存储：文档 / 方案 / memo 的权威载体，由执行会话上传，岗位取回链接回写看板。

GLOSSARY

名词表：面板里也是这些词

词	含义	在哪里体现
项目 Project	一个 Kanban 项目 + 仓库集 + topic 集 + 岗位集	设置 → 项目与岗位
岗位 post	一个 SuperAgent 实例扮演的角色（PM / Worker）	编队页的卡片
岗位职责 duty	岗位正在执行的一张任务	编队页「在飞职责」
宪章 charter	**整个项目一份**、各岗位（含其它 Host）共同遵守的流程与价值流约定；**每周至多一版**	编队页「宪章 vN」徽标
WIP	一个岗位同时在飞的职责数上限	岗位卡片 n / limit
租约 lease	认领时写在评论里的互斥标记	Kanban 任务评论
异常 exception	等人回复 / 阻塞 / 失败 / 六小时无进展	编队页「异常队列」

ONBOARDING

上线准备：一次做完，两个角色共用

STEP 01

装好、连上

```
npx.cmd @deepseek-ai/dsh plugin --profile web add "git+https://git.feg.cn/shawoo/superagent"
```

Windows 用 **npx.cmd**（PowerShell 会拦 .ps1）；macOS / Linux 用 **npx**。装完重启 / 刷新，面板出现「超级智能体」，四个来源（Kanban / Gitea / ntfy / MinIO）ready。

STEP 02

三个来源账号

Kanban：地址 + API key（可留空）+ **项目**（从列表选）+ 监听列（slug）。

Gitea：地址 + 邮箱 + 密码 + **宪章仓库**（只做宪章与职责说明，**不监听 Issue/PR**）；工单要用的仓库由看板卡片的 `repo:` 标签给出。

ntfy：地址 + topic + **入库过滤 tag**；MinIO：端点 + 桶。

STEP 03

岗位账号

Kanban 网页手工建号（**没有建号 API**）：sa-pm-项目 / sa-w1-项目。

记下用户 id；再建 Gitea 账号、约定 ntfy sender。

坑位：看板列名会改，**slug 相对稳定**——监听请填 slug（如 ready,in-progress），不要填显示名。

ONBOARDING · 三件事

上手只要三步：贴钥匙、登记岗位、发布宪章

看板自己说的，面板不重复设置。

STEP 01

贴一把 API key

设置 → 来源 → Kanban：服务器地址 + **API key**（可留空：本机存储记录里就有）→ 保存。

面板随即读出工作区、项目与**看板真实的列**。

STEP 02

登记岗位

从列表选项目（id / 名称 / slug 自动填，主题集默认 `pub_<项目 id>`）+ 每个岗位的 **Kanban 用户 id** + 领哪些列 + 并发上限。

Gitea / ntfy 身份可留空——不是编程任务就不需要。

STEP 03

发布宪章

设置 → 项目与岗位 → 宪章：写当期流程与交付标准 → **发布**。

发布**提交到 Gitea 宪章仓库的** `README.md`（宪章的家）；版本号 +1、记发布者、旧版进历史；**每周至多一版**；工单钉在开工那版。

Kanban 是唯一的直接来源：Gitea / ntfy 的事件先入库成看板上的卡（落在 to-do），PM 指派并移进监听列后，才成为职责。**没有宪章不可开工**：项目在宪章发布前不接受任何任务。

REGISTER

面板里只登记三件事

设置 → 项目与岗位 → 保存。档案里不放密码；保存带版本号校验，别人同时改过会拒绝覆盖。

字段	含义	例子
项目	**从列表选**（这台 Host 的账号能看到的项目）；选中后 id / 名称 / slug 自动填	FDEP（FDE）· ATEAM
宪章	当期怎么做事：流程、职责、交付标准；写完点「发布」——**提交到 Gitea 宪章仓库的 README.md**，一个项目只有一份，**每周至多一版**	version: 12 · publishedBy: pm-rsi
主题集	该项目订阅的 ntfy topic；**留空默认所选项目的** `pub_<项目 id>`	pub_at8x…
Gitea 宪章仓	**Gitea 只做宪章仓库**：宪章与职责说明（`README.md` 同目录的 `duties.md`）在这里；**不监听 Issue/PR**，工单要用的仓库由卡片的 `repo:` 标签给出	shawoo/rsi_superagent
ntfy 入库 tag	只有带这些 tag 的消息才入库成卡（留空=全部）；宿主回复的 tag 是 `dsh-superagent`，不会被回收成卡	dsh
岗位 id / 名称 / 角色	岗位标识与 Team（PM）/ Worker	w1-fdep · SuperAgent Worker 1 (FDEP)
Kanban 用户 id	**决定它领得到哪张卡**	bsFdwgQXqZpYKvD5E2emqQ5TQ81eJ3zG
监听列	该岗位领哪些列的卡（slug）	ready
Gitea 账号 / ntfy sender / 实例	代码与消息身份、跑在哪台机器（都可留空）	sa-w1-fdep · #sa-w1-fdep · web
并发上限 WIP	同时在飞的职责数上限	3

不在这里设的：看板的列（从服务器读）、仓库集（由那张卡自己说：repo:owner/name）、产出类型（由 kind: 标签说）、项目类型。

「编队」页：PM 的主战场

PROJECT CARD

项目卡：宪章 + 岗位

徽标显示宪章版本：**绿色 vN** 本周已发布，**黄色过期** 需要补发。

每个岗位显示 Kanban 身份、实例、**在飞职责 n / limit**；超限会红框提示。

身份显示「未设置」= 岗位还没建 Kanban 账号，领不到卡。

EXCEPTION QUEUE

异常队列：今天先干这个

等人回复 · 阻塞 · 失败 · 长时间无进展。

点条目直达对应工单；每条都带岗位归属，处理完回看板改状态。

概览页也有队伍摘要（岗位数 / 在飞职责 / 异常数 / 宪章新鲜度）。

TWO PERSPECTIVES

两种面板视角，一套后台

右上角切换：**Team 视角**（原 PM 视角）/ **岗位视角**。切换只改变"看到什么、从哪一页开始"——实例、来源、档案、工单队列都不变。

	Team 视角（PM 岗位）	岗位视角
页签	概览 · 编队 · 来源 · 触发 · 评估 · 执行 · 设置	概览 · **我的职责** · **我的岗位** · 来源 · 触发 · 设置（没有"评估"）
编队页	全部项目与岗位 + 异常队列	只有你选的那个岗位 + 我的异常
概览	队伍摘要（岗位 / 在飞职责 / 异常 / 宪章新鲜度）	我的战况（在飞职责 n/limit · 我的异常 · 我的身份 · 宪章）
设置	项目与岗位 · 来源 · 评估 · 执行 · 提示词 · 部署	**我的岗位设置**（只显示并可改自己的岗位）· 来源 · 执行 · 提示词 · 部署
权限	切成 Team 视角前验证 `canManageFlow`；缺权限先弹窗说明账号/角色/缺的权利，仍可切换	不需要管理权限：只干自己领到的活
首页	概览	我的职责

岗位视角的**来源设置只读**（来源由 PM 在 Team 视角配置）。岗位第一次进入时，在「我的岗位」页选一次**你是哪个岗位**（记在这台设备上），之后概览、职责、设置都只围绕它展开。岗位视角里的「**我的岗位设置**」只改自己的岗位身份；项目、topic、宪章由 PM 在 Team 视角维护。**没有项目宪章时，团队页显示「没有宪章不可开工」。**

PM 岗位的日循环

09:00 — **清异常队列**

编队页异常队列置顶处理：等人回复 → 补上下文；阻塞 → 给结论或拆卡；失败 → 重试或改写描述。

10:00 — **看在飞职责与 WIP**

超限 = 先暂停派新卡，判断是卡住了还是上限太小；岗位身份「未设置」就先补账号。

14:00 — **派工**

Kanban 建卡 → 指派给岗位账号 → 放进监听列；不指派 = 不派工。

18:00 — **收工检查**

每张卡都回写了状态与日志才算交付；没回写的不要急着关。收工后到 **清理** 页把已结束职责的工作区集中清掉（有交付物的行要人确认）。

先复盘，再发宪章：流程是长出来的

周复盘 → 新流程

看板不是固定的任务表，而是**一条不断迭代的价值流。**

看这一轮的工作日志 → 哪一列总堵、哪一步没意义 → **推荐新的状态** → 与岗位达成共识 → 确认。

确认后先在 Kanban 里加/改列，再在面板 **设置** → **项目与岗位** → **宪章** 写新版并点「发布」：**提交到 Gitea 宪章仓库的 README.md**（commit 历史即版本历史），其它 Host 读到同一份。

```
version: 12
publishedBy: pm-rsi

# 宪章 · RSI 项目
- 当期流程（列语义与流转）：
  to-do → in-progress → in-review
  → done → qa-验收
- 角色与职责：PM 管流程与异常；Worker 开工先用
  orchestrator_read charter 读当期宪章
- 交付标准：代码 → Gitea PR；非代码 → MinIO
  deliverables/<项目 id>/<ticket id>/<file>
- 实时沟通：项目 topic pub_<项目 id>；
  只有带入库 tag 的消息才入库成卡
- 复盘与改版：每轮闭环后复盘，宪章每周至多一版
```

面板编队页会把**看板的列**与**宪章声明的列**并排显示：出现黄色「看板多出的列」= 流程长出来了，还没被确认。编队页的宪章徽标会显示**载体**（如 `shawoo/rsi_superagent/README.md`）。**每周至多一版**：7 天内再发会被拒并告知下次可发布时间；确需提前改，点「仍然发布（覆盖每周一次的限制）」。

VALUE STREAM

看板是 VSM：流程与产出都不是固定的

岗位的任务**不必然是编程任务**，看板也不是固定的列。

PROJECT KIND

项目类型

软件交付 / 销售机会 / 合同管理 / 客诉处理 / 物流投递 / 其他。

类型决定**产出长什么样**，不是所有卡都要代码。

WORKFACE

工作面

仓库只是工作面之一：文档空间、CRM、表单、看板本身。

非软件岗位**不需要 Gitea 身份**，有 Kanban 身份就能接单。

DELIVERABLE

产出类型是开放的

卡片用 **kind:<类型>** 标签声明产出：doc、quote、contract、complaint…

权威载体只有一个，但**不限于代码。**

⚠ 看板自己的自动化会很快把卡带走（实测：进入 ready 后约 20 秒被搬到 in-progress）——监听列与轮询间隔要跟看板流程的速度匹配；没被接住时，再推一次。

WORKER · CLAIM

接单：只领自己的，并让互斥成立

Kanban 没有 CAS，所以认领必须自己保证互斥。

STEP 01

读回校验

把 assignee 读回来确认是自己；**别人的卡不代领、不自领。**

STEP 02

写租约

评论追加 **lease::<岗位id>**，token 用接单时刻。

```
lease:20261104T1730:w1-fdep
```

STEP 03

仲裁

同一张卡多条租约时，**createdAt 最早**的那条获胜，其余退出。

完成后：状态改到 done（按约定列），释放租约，交付物写进评论。

执行与交付：产出有唯一权威载体

产出类型	权威载体	回写看板
代码变更	Gitea PR（一仓一目录一 PR）	PR 链接 + 评审状态
文档 / 方案	MinIO 对象（「MinIO 对象存储」来源卡：端点/桶/凭据，默认取存储记录）	对象键 + URL
memo / 决策	MinIO 对象（长）或任务评论（短）	摘要 + 结论
工作日志	Kanban 任务评论（追加式）	即为看板内容

交付前自查

- ✓ 只改了被授权的仓库与目录
- ✓ 产出有可点击引用
- ✓ 工作日志已追加
- ✓ 状态已更新、租约已释放

边界

开工第一步：用 `orchestrator_read charter` 读当期宪章（返回版本与载体），不需要 shell/HTTP。

不跨工作目录交付；需求有歧义**不要猜**：在任务里提问并转 blocked，进入 PM 的异常队列。

文档/方案**存 MinIO，不贴进评论**：`minio_write` 到 `deliverables/<项目 id>/<ticket id>/<file>` → `minio_read` 回读确认 → 报告写 `bucket/key`；卡片只留进度与结果。

看板由宿主写：岗位报告 ready，宿主把结果写成评论并把卡片推进交付列；岗位不要自己调用看板 API 搬卡。

ntfy：项目 topic 只有带入库 tag（如 `dsh`）的消息才入库成卡；没带的当聊天，不产生卡。

CLEANUP

收工之后：集中清理工作区

每个职责都会留下一个工作区和一条 Session；清理是 PM 的收尾动作。

WHERE

「清理」页（PM 视角）

只列**已结束**职责留下的工作区：按大小排序，标出 Session 数、顶层文件、**是否有交付物**。

未结束的职责、被别的职责引用的目录，一律跳过。

HOW

先看，再勾，再确认

默认只勾「已结束 + 可清 + 无交付物」；**有交付物的行必须人点**。

要连目录一起删，单独勾「同时删除工作区文件夹」并在弹窗里确认——不可恢复。

WHAT STAYS

Session 只解除登记

没有受支持的 Session 删除 API：清理只**解除工作区登记**，日志保留可回放。

stateDir、执行根、盘根与指向执行根之外的符号链接都会被拒绝。

这就是「工作区列表膨胀」的解药：跑得越久，已结束的职责越多，定期清一次比到处找目录省事。

EXCEPTIONS

异常处置表

等人回复

职责在等人。检查提问是否清楚，必要时补上下文或改派。

阻塞

缺权限、缺信息还是外部依赖？给结论，或把卡拆小。

失败

看日志后重试；若反复失败，多半是任务描述不清，改写再派。

长时间无进展

六小时没有更新。ntfy 私聊岗位；仍无响应则人工接手。

KNOWN LIMITS

会撞到的限制（别浪费力气）

限制	现象	绕法
限流 429	大量读写时 Kanban 返回 429	退避重试、错开批量写
突发 401	偶尔一分钟内鉴权失败	等一分钟，脚本已内置退避
看板截断	不带 status 查询会按列截断，静默漏任务	永远带 ?status= 与 limit=100
assignee 单值	一个人只能一个 assignee	多人合卡用评论标角色（agent:pm/ux/bi/api）
账号只能 UI 建	Kanban 无建号/发 key API	在 Kanban 网页手工建号
MinIO 边界	岗位（host）不直接写桶	执行会话用 `minio_*` 工具上传，key 固定为 `deliverables/<项目 id>/<ticket id>/<file>`
宪章过期 / 每周上限	超过 7 天没更新；或 7 天内想再发一版	周复盘后发；确需提前改，点「仍然发布（覆盖每周一次的限制）」并在正文说明理由
看板自动化抢跑	进入 ready 约 20 秒就被搬到 in-progress，60 秒轮询会漏	轮询间隔 ≤ 15 秒，或把 in-progress 也纳入监听列
宿主重启打断工单	跑长任务时保存来源设置会重启插件，正在跑的工单被中断	工单会带「host restarted」输入自动续跑；长任务期间不要改设置
ntfy 换 topic	读游标跨 topic 保留，切换瞬间的消息可能被跳过	换 topic 自动回退游标 600 秒（入库标记去重）
Gitea 只做宪章	Gitea 卡片上没有事件仓库，也不监听 Issue/PR	工单需要代码仓库时，在卡片打 `repo:owner/name`；宪章仓留空则退回 Kanban 描述镜像

GO-LIVE

上线检查清单

PM 岗位

- ✓ 四个来源 status 都 ready（Kanban / Gitea / ntfy / MinIO）
- ✓ 项目 **列表** 选好（主题集默认从 `pub_<项目 id>`），岗位已登记（监听列 / Kanban 用户 id）
- ✓ 每个岗位的 Kanban 用户 id 已填写
- ✓ 宪章 v1 已 **提交到 Gitea 宪章仓库的 README.md**（编队页徽标显示 carrier=gitea）
- ✓ 切到 Team 视角时权限自检通过（admin，或已知缺权限仍要切换）
- ✓ 派过一张卡，并看到它被领走
- ✓ 异常队列能正常显示；ntfy 入库 tag 已配置，MinIO 卡片桶可读

岗位（Worker）

- ✓ Kanban 账号已建，卡确实指派给自己；开工先用 `orchestrator_read charter` 读宪章
- ✓ Gitea 测试连接通过
- ✓ ntfy topic / sender 已配置，允许名单里有自己
- ✓ 认领时做了读回 + 租约评论
- ✓ 产出带引用，并回写了状态与日志

载体分工：宪章 / 任务 / 消息 / 资产

一个载体一件事；凭据默认取本机存储记录（Masked Key），卡片可逐字段覆盖。

环节	载体	规矩
宪章	Gitea「**宪章仓库**」的 `README.md`（与职责说明 `duties.md` 同仓）	发布 = 提交该文件；commit 历史 = 版本历史；编队页徽标显示载体
任务	Kanban 看板（监听列可配，如 `to-do`）	卡片是唯一的派工来源；看板列就是价值流
实时消息	ntfy 项目 topic `pub_<项目 id>`	只有带**入库 tag**（如 `dsh`）的消息才入库成卡；宿主回复 tag `dsh-superagent` 不会被回收
数据资产	「MinIO 对象存储」来源卡（桶 `ateam`）	非代码交付物：`deliverables/<项目 id>/<ticket id>/<file>`，卡片只留进度与结果

四张来源卡的凭据默认来自本机存储记录（`/BDD/API/AI/dsh/storage/<service>`）：**留空 = 记录值，填写 = 覆盖**；密钥只在宿主内存，不写进设置文件。

CHARTER GATE

没有宪章 不可开工

项目档案里的项目，在宪章发布前（版本 0）不接受任何任务。

两道闸门都在主机侧

- ✓ **评估接受**：accept 变成带 `noCharter` 理由的 ignore——不建工单。
- ✓ **主会话显式派工**（`orchestrator_route`）：同样拒绝，返回「Ticket was not created」。
- 不属于任何项目的事件不设闸门；面板团队页对缺宪章的项目显示红色提示。

岗位怎么拿到宪章

- ✓ 开工第一步：`orchestrator_read charter`，返回 `version` / `carrier` / 正文。
- ✓ 不需要 shell / HTTP，也不需要 Gitea 凭据：宿主读完再交给岗位。
- 没有宪章是 PM 要修的事（Team 视角 → 项目与岗位 → 发布宪章），不是岗位绕过的理由。

VALUE STREAM · MEASURED 2026-11-06

一次值流闭环：4 分 40 秒

宪章 v11（Gitea README）→ 卡进 RSI 项目 → ntfy 入库 → 工单 → MinIO 交付 → 卡片 done。

时刻	事件	证据
22:14:36	读宪章 v11	`carrier=gitea · shawoo/rsi_superagent/README.md`
22:14:37	派卡进 `to-do`，ntfy 通知发出	卡 #12，指派 PM 岗位账号
22:14:46	ntfy 通知入库成卡	**9 秒**，标签 `from:ntfy`
22:14:56	工单创建并派工	**19 秒**，钉住 charter v11
22:17:17	MinIO 交付物出现	**2 分 41 秒**，对象回读 UTF-8 干净
22:19:16	卡片到 `done`	**4 分 40 秒**，评论含 `bucket/key`

复盘发现并修掉：① 岗位读不到 Gitea 宪章 → 新增 `orchestrator_read charter`；② ntfy 换 topic 可能跳过消息 → 游标回退 600 秒；③ 交付物 key 两种写法 → 只留 `deliverables/` 一种。→ 改进写进 SOP §3.6，宪章发布 v12。

NEXT

先跑一个项目，一个岗位

登记档案 → 派一张卡 → 看它被领走 → 看它回写。
剩下的照着 §3、§4 循环。

SOP.md • 完整流程

RD.md • 叙事与未尽事项

TDD/008 • 实测基线

面板
编队 • 设置 → 项目与岗位

命令
dsh-superagent status

演练
scripts/kaneo-story-probe.mjs

GO